

在学习过程中，学到的知识点，容易忘记的

js部分

1. 执行上下文对象

全局执行上下文对象：是window对象，形成于js代码执行之前，所有的全局变量和全局函数都是window对象的属性和方法。在关闭页面时被销毁。

函数执行上下文对象：首先并没有接口可以访问，必须要**调用函数**才会被创建，每个函数都有自己的函数执行上下文对象,调用结束后会被销毁。

2. 全局预处理和函数预处理

全局：1.var声明的全局变量添加为window对象的属性，值为undefined。2.将使用function关键字声明的全局函数添加为window的方法，值为函数体

函数：1.var声明的局部变量添加为函数执行上下文对象的属性，值为undefined。

2.将实参赋值给形参，并添加为函数执行上下文对象的属性

3.将使用function关键字声明的局部函数添加为函数执行上下文对象的方法，值为函数体。（**按照执行顺序**）

3. 闭包

产生闭包的原因：让我们可以在函数外部读取函数内部的局部变量

闭包形成所需要具备的条件

- 1、嵌套函数
- 2、内部函数要使用外部函数的变量(函数)
- 3、要在外部函数中使用内部函数
- 4、要调用外部函数

*/

```
function fn1() {  
  var a = 1  
  function fn2() {  
    return a  
  }  
  console.dir(fn2)  
  return fn2  
}  
var f = fn1()  
console.log(f());
```

4. 作用域

作用域

作用域 (scope) 指的是变量存在的范围。在 ES5 的规范中, JS 只有两种作用域: 一种是全局作用域, 变量在整个程序中一直存在, 所有地方都可以读取; 另一种是函数作用域, 变量只在函数内部存在。ES6 又新增了块级作用域, 暂时不涉及。

没有在任何函数内部声明的变量是全局变量, 在任何函数内部都可以获取全局变量并更改

函数内部通过var声明的局部变量, 在全局是无法获取的

代码写好, 作用域就确定了

函数内部非var关键字声明的变量, 为全局变量

没有块级作用域

如果有函数嵌套, 则会由内而外形成作用域链

```
*/  
// 什么是块级作用域  
console.log(a); // 执行上下文对象添加变量的时候并不关系条件是否成立  
if (false) {  
  var a = 1  
}
```

5. 回调函数

将一个函数作为参数传入到另外一个函数中, 并且在另外一个函数中调用, 那么被传入的这个函数就称为回调函数。

6. 递归

7. 面向对象编程

1. 构造函数

面向对象编程

函数式编程

什么是面向对象？

编程的本质实际上是在根据不同的业务需求处理数据。所以我们就可以把这些不同的业务抽象成各个对象。各个对象都有自己的特征(属性)和行为(方法)。属性中保存了需要被处理的数据，而行为可以根据不同的业务需求来修改数据。

怎么生成对象？

这里我们要采用面向对象的思想来生成对象。对象实际是由提前被抽象出来模板(构造函数)生成(实例化)的。

什么是构造函数？

构造函数的声明从本质上来讲，和普通函数没有差别，从形式上来讲，构造函数的首字母推荐使用大写字母。从使用上来讲，使用new命令，后面跟着函数的调用，这样的函数就被叫做构造函数，是用来生成对象的。

2.new命令和this指向

this指向：

```

<script>
  /*
    new命令
    使用new命令时，它后面的函数依次执行下面的步骤。
    1. 创建一个空对象，作为将要自动返回的对象。
    2. 将这个空对象的原型指向构造函数的`prototype`属性
    3. 将空对象赋值给函数内部的`this`关键字
    4. 开始执行构造函数内部的代码
  */
  // this始终指向一个对象，是当前属性或方法所在的对象。
  // this是一个动态的值，是跟使用函数的方式有关的
  function Person() {
    this.userName = 'xiaoming'
    console.log(this);
  }
  var p1 = new Person() // 实例化构造函数生成对象，函数中的
  // this就是实例化后的对象
  // console.log(p1);

  // var p2 = Person()
  var p2 = window.Person() // 没有使用关键字new，而是直接调用
  // 函数，相当于通过window.方法的方式执行函数。this就会指向函数的
  // 调用方，也就是window。
  // console.log(p2); // undefined

```

new命令：

```

<script>
// 使用new命令实例化构造函数
/* function Person() {
  this.userName = 'xiaoming'
  // return true // 如果返回的是非对象数据，则会被忽略，直接返回this对象
  return {
    userName: 'xiaohua'
  } // 如果返回的是对象，则此返回的对象会把this对象覆盖掉
}
var p1 = new Person()
console.log(p1); */

// 没有使用this的函数，如果返回的是非对象数据，返回的则是一个空对象
function Person() {
  return 1
}
var p1 = new Person()
console.log(p1);

```

// 总结: new命令总是返回一个对象，要么是实例对象，要么是函数内return语句指定的对象。

new.target: 让不是构造函数的调用，变成构造函数


```
// new.target
function Person(userName) {
  // console.log(new.target);
  if (!new.target) {
    return new Person(userName)
  }
  this.userName = userName
}

var p1 = Person('xiaoming')
console.log(p1);
var p2 = new Person('xiaohua')
console.log(p2);
```

this的由来:

函数在声明的时候, 作用域就已经确定了。

为什么要设计this:

<title>04-为什么要设计this</title>

</head>

<body>

<script>

```
var a = 2
```

```
function fn() {
```

```
  console.log(this.a); // js中一切皆对象, 运行环境也是对象, 所以函数都是在某个对象下运行的, this就是指函数运行时所在的对象(环境), 所以this是动态的, 在调用之前是不确定的, 只有在调用的时候才可以确定this的指向。
```

```
}
```

```
var obj = {
```

```
  a: 1,
```

```
  fn: fn
```

```
}
```

```
obj.fn()
```

```
fn()
```

```
var obj2 = {
```

```
  a: 3,
```

```
  fn: fn
```

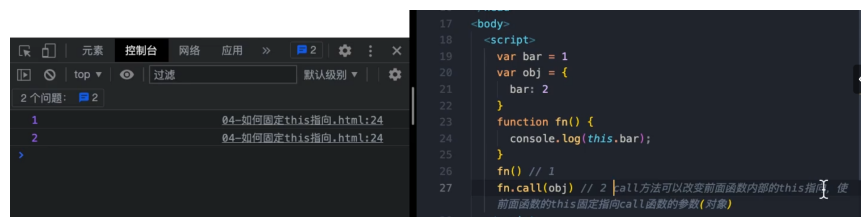
```
}
```

```
obj2.fn()
```

this使用场合:

固定this的指向:

call方法:



```
17 </body>
18 <script>
19   var bar = 1
20   var obj = {
21     bar: 2
22   }
23   function fn() {
24     console.log(this.bar);
25   }
26   fn() // 1
27   fn.call(obj) // 2 [call方法可以改变前面函数内部的this指向，使
   前面函数的this固定指向call函数的参数(对象)]
28 </script>
```

```
<title>04-如何固定this指向</title>
</head>
<body>
  <script>
    var bar = 1
    var obj = {
      bar: 2
    }
    function fn() {
      console.log(this.bar);
    }
    fn() // 1
    fn.call(obj) // 2 call方法可以改变前面函数内部的this指向，使前面函数的this固定指向call函数的参数(对象)

    // call方法的参数，应该是一个对象，如果参数为空、null、undefined，则默认传入全局对象window
    fn.call()
    fn.call(null)
    fn.call(undefined)
    fn.call(window)
```


//回调函数中的this指向问题

```
var obj2 = {  
  bar: 1,  
  fn: function () {  
    console.log(this.bar);  
  },  
};  
obj2.fn(); //1
```

```
function fn2() {  
  obj2.fn.call(obj2);  
}  
var bar = 2;  
function fn(f) {  
  f();  
}  
f(fn2); //1
```

```
var obj3 = {  
  bar: 3,  
  fn: function (f) {  
    f();  
  },  
};  
obj3.fn(fn2); //1
```

call(接收参数):

```
var obj = {  
  x: 1,  
  y: 2  
}  
function fn(x, y) {  
  console.log(x, y);  
  console.log(this.x + this.y + x + y);  
}  
fn.call(obj, 10, 16) // call还可以接收更多的参数, 从第二个开始往后的参数都会传递到call前面的函数中, 作为函数的参数。
```

call转换类数组对象为数组

```
// call转换类数组对象为数组
var obj = {
  0: 'a',
  1: 'b',
  2: 'c',
  length: 3
};
// var arr = ['a', 'b', 'c', 'd']
// var result = arr.slice()
// console.log(result);
var result = [].slice.call(obj)
console.log(result);
```

apply方法：将数组中的空属性转变成undefined（使用array和foreach方法）

```

// 将数组的空属性变为undefined
// console.log(Array('a', 'b', 'c'));
var arr = ['a', 'b', 'c', , , 'd']
// console.log(arr);
var arr2 = Array.apply(null, arr)
// console.log(arr2);

// forEach方法: 是一个处理数组的方法, 可以通过 数组.forEach(回调函数) 接收一个回调函数, 会依次将数组的元素通过回调函数的参数传给回调函数, 可以在回调函数中做进一步处理。
/* var arr3 = ['aa', 'bb', 'cc']
arr3.forEach(function (v) {
  console.log(v);
}) */

// forEach方法会默认跳过空属性, 但是不会跳过undefined
arr.forEach(function (v) {
  console.log(v);
})
arr2.forEach(function (v) {
  console.log(v);
})

```

bind方法: 会返回一个新函数, 且不进行调用, 而call和apply会调用该函数。

```
/*
  bind方法:
  bind方法用于将函数体内的this绑定到某个对象上, 并返回一个新
  函数。且不进行调用。call和apply都会调用该函数。
*/
var bar = 1
function fn() {
  console.log(this.bar);
}
var obj = {
  bar: 2
}
fn()
fn.call(obj)
var newFn = fn.bind(obj)
newFn()
// 实用场景, 只需要改变函数内部的this指向, 而不需要立即执行, 所
// 以可以先返回一个新函数, 在合适的时候调用。
```

```

    return x * this.m + y * this.n
}
var obj = {
  m: 2,
  n: 3
}
var newAdd = add.bind(obj, 5, 6)
console.log(newAdd());
var newAdd2 = add.bind(obj, 5)
console.log(newAdd2(6)); */

var obj = {
  bar: 'obj',
  arr: [1, 2, 3],
  fn: function () {
    this.arr.forEach(function (v) {
      console.log(v);
      console.log(this.bar);
    }).bind(this)
  }
}
obj.fn()

```

8. js原型链

01-prototype对象

```

/*
  对象的继承：
  面向对象编程很重要的一个方面，就是对象的继承。A对象通过继承B对
  象，就能直接拥有B对象的所有属性和方法。这对于代码的复用是非常有
  用的。
*/

```

基于原型 (prototype) 的继承

大部分面向对象的编程语言，都是通过“类” (class) 实现对象的继承。JS语言的继承不通过 class，而是通过“原型对象” (prototype) 实现，ES6当中有“类”的继承，但是本质还是过“原型对象” (prototype) 实现。

*/

</script>

// 各属性值不同是可以接受的，因为不同实例对象的属性值基本是不同的。可是方法却是一样的，可是方法也不能在不同实例之间共享，每个每生成一个实例对象，就要创建一个同样行为的方法，shi这样就造成了资源浪费。

<script>

```
//构造函数的缺点:属性可以是不同，可以分别保存，
//虽然两个对象的函数体完全相同，但实际上保存的是函数的地址
//所以两个对象都各自保存了这个函数
// 所以相同的方法却不是被p1,p2两个对象共享的，方法不能被不同的实例共享，这会造成资源的浪费，
//
// function Person(name, age) {
```

//通过prototype添加方法

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}
Person.prototype.eat = function () {
  console.log("吃饭");
};
var P1 = new Person("xiaoming", 16);
var P2 = new Person("xiaohua", 20);
console.log(P1);
console.log(P2);
P1.eat();
P2.eat();
console.dir(Person);
//隐式原型和显示原型
console.dir(P1.__proto__);
console.dir(Person.prototype);
/*原型对象的属性和方法，都可以被实例对象所共享，不仅节省的内存，还体现了实例对象之间的联系*/
```

```
1
2 <!DOCTYPE html>
3 <html lang="en">
4   <head>
5     <meta charset="UTF-8" />
6     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7     <title>01-prototype属性</title>
8   </head>
```



```
9   <body>
10   <script>
11       //构造函数的缺点:属性可以是不同,可以分别保存,
12       //虽然两个对象的函数体完全相同,但实际上保存的是函数的地址
13       //所以两个对象都各自保存了这个函数
14       // 所以相同的方法却不是被p1,p2两个对象共享的,方法不能被不同的实例共享,这会造成资源的浪费,
15       //
16       // function Person(name, age) {
17       //     this.name = name;
18       //     this.age = age;
19       //     this.eat = function () {
20       //         console.log("吃饭");
21       //     };
22       // }
23       // var P1 = new Person("xiaoming", 16);
24       // var P2 = new Person("xiaohua", 20);
25       // console.log(P1);
26       // console.log(P2);
27
28       /*prototype属性:
29       // 每个函数都有一个prototype属性(对象才有属性,所以相当于把函数作为一个对象来看),指向一个
对象*/
30       // function fn() {}
31       // console.dir(fn);
32       // fn.prototype.abc = "123";
33       // console.log(fn.prototype);//对于普通函数,此属性没什么用
34       // //确定,fn是一个对象
35       // console.log(typeof fn.prototype);//object
36
37       //通过prototype添加方法
38       function Person(name, age) {
39           this.name = name;
40           this.age = age;
41       }
42       Person.prototype.eat = function () {
43           console.log("吃饭");
44       };
45       var P1 = new Person("xiaoming", 16);
46       var P2 = new Person("xiaohua", 20);
47       console.log(P1);
48       console.log(P2);
49       P1.eat();
```

```

50     P2.eat();
51     console.dir(Person);
52     //隐式原型和显示原型
53     console.dir(P1.__proto__);
54     console.dir(Person.prototype);
55     /*原型对象的属性和方法，都可以被实例对象所共享，不仅节省的内存，还体现了实例对象之间的联系*/
56
57     //原型对象上的属性和方法，不是实例对象自己的属性（方法）。只要修改原型对象，变动立刻就会体现在所有的实例对象上。
58     Person.prototype.eat = function () {
59         console.log("吃饭！");
60     };
61     P1.eat();
62     P2.eat();
63     //在原型对象上添加属性
64     Person.prototype.position = "student";
65     console.log(P1.position, P2.position);
66
67     /*总结：
68     原型对象的作用：就是用来定义共享的属性和方法，但是一般用来共享方法。而实例对象可以看作原型对象衍生出来的子对象。
69     */
70     </script>
71 </body>
72 </html>
73
74

```

02-初识原型链：

```

1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8" />
5          <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6          <title>02-初始原型链</title>
7      </head>
8      <body>
9          <script>
10             /*
11             原型链：

```

```

12     JS规定，所有对象都有自己的原型对象，函数也是对象，所以也不例外。
13     函数实际是由构造函数Function实例化来的
14     1.任何对象都可以充当其他对象的原型对象
15     2.原型对象也有自己的原型对象
16     因此就会形成一个原型链(prototype-chain)
17     */
18     //对象上有隐式原型，又因为隐式原型全等于显式原型，所以对象是通过隐式原型来访问到函数添加到显
    式原型上的属性和方法。
19     // var fn = new Function();
20     // console.dir(fn);
21     // Function.prototype.abc = "abc";
22     // console.log(fn.abc);
23
24     function Fn() {}
25     var f1 = new Fn();
26     console.log(f1.__proto__ === Fn.prototype); //true
27     console.log(Fn.__proto__ === Function.prototype); //ture
28     console.log(Fn.prototype.__proto__ === Object.prototype); //true
29
30     //所有对象都继承了object.prototype的属性
31     console.log(Object.prototype.__proto__); //null原型链的尽头。
32
33     //Object.getPrototypeOf方法：返回参数对象的原型。
34     console.log(Object.getPrototypeOf(f1) === Fn.prototype); //true
35     </script>
36 </body>
37 </html>
38

```

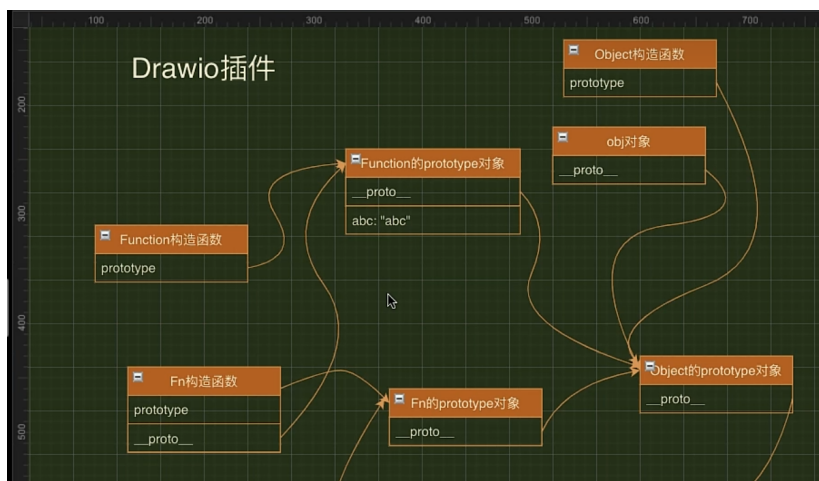
用构造函数Function实例化fn函数:

```

▼ f anonymous ( ) ⓘ
  length: 0
  name: "anonymous"
  ▶ prototype: {}
  arguments: null
  caller: null
  [[FunctionLocation]]: VM6:1
  ▶ [[Prototype]]: f ()
  ▶ [[Scopes]]: Scopes[1]

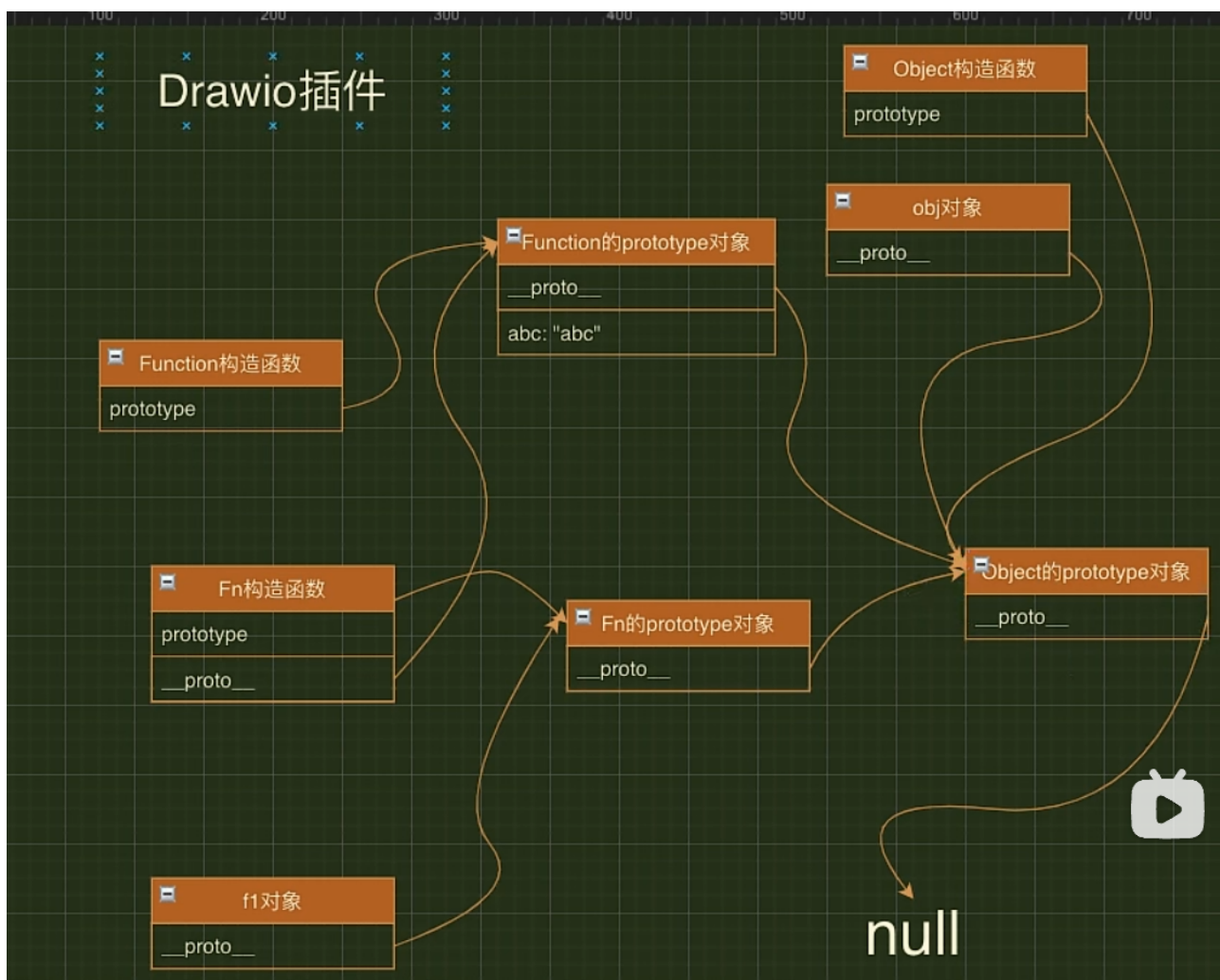
```

fn作为函数时有显式原型，作为对象时有隐式原型



fn作为对象时有隐式原型，指向Function构造函数的prototype对象，而Function构造函数有一个prototype属性，指向他的prototype对象，所以fn和Fn都指向同一个prototype地址，所以隐式原型的对象可以访问到abc属性。

原型链：



03-属性的查找过程

```

1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />

```

```

5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <title>03-属性的查找过程</title>
7 </head>
8 <body>
9     <script>
10         function Fn(a) {
11             this.a = a;
12         }
13         var f1 = new Fn("a");
14         console.dir(Fn);
15         console.dir(f1);
16         console.dir(f1.a); //a
17         Fn.prototype.a = "aa";
18         console.log(f1.a); //a
19         //如果对象自身和他的原型，都定义了一个同名属性，那么优先读取对象自身的属性，这叫做覆盖
        (overriding)
20     </script>
21 </body>
22 </html>
23

```

9. 手写new命令函数（不是很懂，call, apply, band方法忘记了，看一下）

```

10.
11. 1
12. 1
13. 1
14. 1
15. 1
16. 1
17. 1
18. 2
19. 1
20. 1
21. 1
22. 1
23. 1
24. 11

```

25. 1

26. 1

27. 11

28. 1

29. 1

30. 1

31. 1

32. 1